

1. What is Type Conversion?

Type conversion means converting a value from one data type to another data type.

For example:

```
int num = 10;
double result = num;
```

Here, the `int` value `10` is converted to `double`.

C++ supports two common types of conversion:

1. **Implicit Type Conversion**
2. **Explicit Type Conversion**

2. Implicit Type Conversion

Implicit type conversion occurs automatically. The programmer does not need to write any casting instruction.

Example

```
int num = 10;
double result = num;

cout << result;
```

Output:

```
10
```

C++ automatically converts `int` to `double`.

Common Implicit Conversions

From	To	Example
char	int	'A' → 65
char	float	'A' → 65.0
char	double	'A' → 65.0
bool	int	true → 1
bool	float	true → 1.0

From	To	Example
bool	double	true → 1.0
int	float	10 → 10.0
int	double	10 → 10.0
float	double	10.5 → 10.5
int	char	65 → 'A'*

Important: float → double

float → double is normally an **implicit conversion**.

```
float num = 12.5f;  
double result = num;
```

C++ automatically performs the conversion.

3. char to int

When a char is converted to an int, its **ASCII value** is obtained.

```
char ch = 'A';  
int num = ch;  
  
cout << num;
```

Output:

```
65
```

Because:

```
'A' = 65  
'B' = 66  
'C' = 67
```

4. int to char

An integer can also be converted to a character.

```
int num = 65;  
char ch = num;  
  
cout << ch;
```

Output:

A

Here, 65 is the ASCII value of A.

Important Point

`int` → `char` can be an implicit conversion.

However, the integer should represent a meaningful character code.

For ASCII:

```
65-90  → A-Z  
97-122 → a-z  
48-57  → 0-9
```

For example:

```
int num = 65;  
char ch = num;
```

produces:

A

But an arbitrary value such as 500 should not be expected to produce a meaningful standard ASCII character.

5. Explicit Type Conversion

Explicit type conversion means that the programmer manually tells C++ to convert a value from one data type to another.

Basic Syntax

One common beginner-friendly syntax is:

```
(data_type)value
```

Example

```
double num = 10.75;

int result = (int)num;

cout << result;
```

Output:

```
10
```

The programmer explicitly converts `double` to `int`.

6. Why Do We Need Explicit Conversion?

Explicit conversion is especially useful when converting from a type that can contain more information to a type that can contain less information.

For example:

```
double num = 25.75;
int result = (int)num;
```

The result is:

```
25
```

The decimal part `.75` is removed.

This is called **loss of information**.

7. Common Explicit Conversions

From	To	Example
double	int	10.75 → 10
float	int	10.75 → 10

From	To	Example
double	float	10.75 → 10.75
float	char	65.5 → 'A'*
double	char	65.5 → 'A'*
int	char	65 → 'A'
int	bool	5 → true

- The numeric value is first converted to an appropriate integer value before the character conversion.

8. Explicit `int` → `char`

Although `int` → `char` can happen implicitly, we can also perform it explicitly.

```
int num = 65;

char ch = (char)num;

cout << ch;
```

Output:

```
A
```

Here, `(char)` tells C++:

Convert this integer value to `char`.

9. Explicit `double` → `int`

```
double num = 15.89;

int result = (int)num;

cout << result;
```

Output:

```
15
```

The fractional part `.89` is discarded.

10. Explicit `float` → `int`

```
float num = 12.75f;

int result = (int)num;

cout << result;
```

Output:

```
12
```

11. Explicit `double` → `float`

```
double num = 25.75;

float result = (float)num;

cout << result;
```

Here, the programmer explicitly requests conversion from `double` to `float`.

12. `bool` Conversion

A `bool` has two values:

```
true  → 1
false → 0
```

`bool` → `int`

```
bool status = true;

int result = status;

cout << result;
```

Output:

```
1
```

This can happen implicitly.

`int` → `bool`

```
int num = 5;

bool result = num;

cout << result;
```

Output:

```
1
```

In a Boolean context:

```
0          → false
non-zero → true
```

13. Implicit vs Explicit Conversion

Feature	Implicit	Explicit
Who performs conversion?	C++ automatically	Programmer
Casting instruction required?	No	Yes
Example	<code>double d = num;</code>	<code>int n = (int)d;</code>
Programmer controls conversion?	No	Yes
Useful for avoiding unwanted conversion?	Limited	Yes

14. Important Examples

Example 1: Implicit

```
int num = 10;
```

```
double result = num;
```

C++ automatically converts:

```
int → double
```

Example 2: Explicit

```
double num = 10.75;  
int result = (int)num;
```

The programmer manually converts:

```
double → int
```

Result:

```
10
```

Example 3: Character and ASCII

```
char ch = 'A';  
int num = ch;
```

Result:

```
65
```

This is an implicit conversion.

Example 4: Integer and Character

```
int num = 65;  
char ch = num;
```

Result:

```
A
```

This can also be an implicit conversion because C++ allows the numeric-to-character conversion.

Example 5: Explicit Type Casting in Arithmetic

Explicit type casting can be used to convert a value to another data type before an arithmetic operation.

Consider the following expression:

```
cout << 5 / 2;
```

Here, both 5 and 2 are integers (`int`).

Therefore, C++ performs **integer division**:

```
5 / 2 = 2
```

Using Explicit Type Casting

We can explicitly convert 5 from `int` to `float`:

```
cout << (float)5 / 2;
```

Here:

```
(float)5
```

means:

Convert the integer value 5 to a `float`.

The expression is now treated as:

```
5.0 / 2
```

Since one value is a `float`, C++ performs **floating-point division**.

```
5.0 / 2 = 2.5
```

Output

```
2.5
```

Compare the Two Expressions

```
cout << 5 / 2;  
cout << (float)5 / 2;
```

Results:

```
5 / 2      → 2  
(float)5 / 2 → 2.5
```

Key Point

Explicit type casting allows us to control the data type used in an expression.

In this example, `(float)` changes integer division into floating-point division.

Another Example

```
int a = 5;  
int b = 2;  
  
cout << (float)a / b;
```

Output:

```
2.5
```

Here, only `a` is explicitly converted to `float`. Because one operand is a `float`, the division produces a floating-point result.

15. Easy Conversion Pattern

A useful pattern to remember is:

```
char → int → float → double
```

These conversions generally move toward types capable of representing a wider range or more precision.

The reverse direction can require explicit conversion and may cause information loss:

```
double → float → int
```

For example:

```
10.75 double
  ↓
10 float
  ↓
10 int
```

When converting a floating-point value to an integer, the fractional part is discarded.

16. Key Points to Remember

1. **Implicit conversion** is performed automatically by C++.
2. **Explicit conversion** is performed manually by the programmer.
3. `float` → `double` is normally **implicit**.
4. `char` → `int` can be implicit and gives the character's numeric code.
5. `int` → `char` can also be implicit.
6. `65` → `'A'` because `65` is the ASCII value of `A`.
7. `double` → `int` can lose the decimal/fractional part.
8. `0` → `false` and a non-zero value → `true`.
9. A conversion can be possible **both implicitly and explicitly**.
10. The main difference is **who performs the conversion: C++ or the programmer**.

Quick Revision

IMPLICIT

C++ does the conversion automatically.

Example:

```
int num = 10;
double d = num;
```

EXPLICIT

Programmer tells C++ to perform the conversion.

Example:

```
double d = 10.75;
int num = (int)d;
```